

Documentation Technique - Système Avatar AI

Waka Voice Burkina - Azure AI Avatar Integration

Version: 1.0 **Date:** 19 Novembre 2025 **Auteur:** Équipe Technique Waka **Temps de développement:** 140 heures

Table des Matières

- 1. Vue d'ensemble du Système
- 2. Architecture Globale
- 3. Composants Frontend
- 4. Composants Backend
- 5. Intégration Azure Services
- 6. Système de Configuration
- 7. Gestion des Outils (24 Tools)
- 8. WebSocket et Temps Réel
- 9. Base de Données Cosmos DB
- 10. Sécurité et Authentification
- 11. Optimisations et Performance
- 12. Défis Techniques Résolus
- 13. Tests et Validation
- 14. Déploiement et CI/CD
- 15. Maintenance et Évolution

1. Vue d'ensemble du Système

1.1 Objectif du Projet

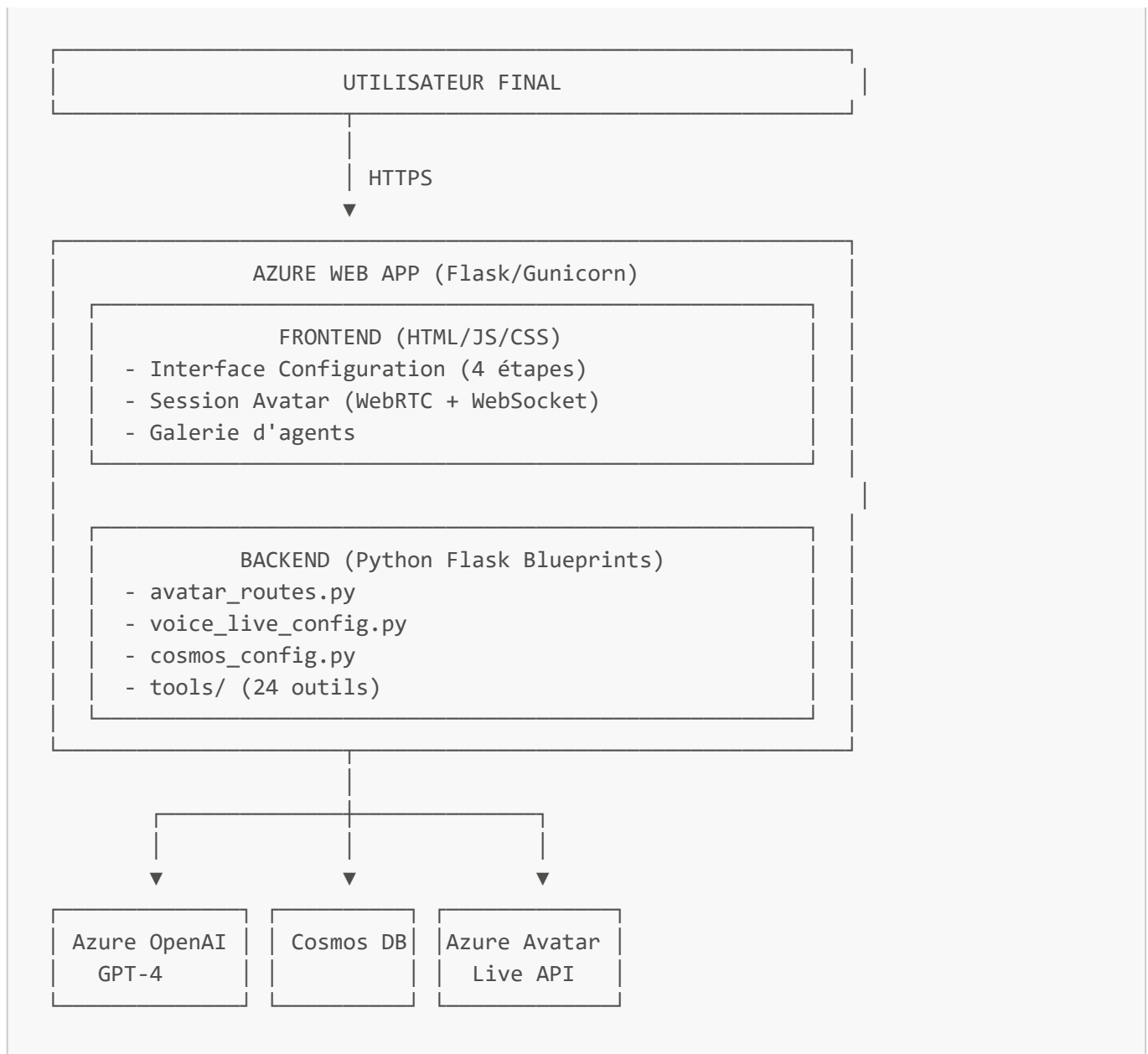
Le système Avatar AI de Waka Voice Burkina est une plateforme complète permettant la création, la configuration et la gestion d'agents conversationnels avec avatars vidéo en temps réel. Le système intègre :

- **Azure AI Avatar** : Streaming vidéo en temps réel d'avatars parlants
- **Azure OpenAI GPT-4** : Intelligence conversationnelle avancée
- **Azure Personal Voice** : Clonage vocal personnalisé
- **24 Outils métiers** : Intégration de services variés (météo, email, CV, réservations, etc.)
- **Cosmos DB** : Stockage distribué et haute disponibilité

Temps total de développement : 140 heures

2. Architecture Globale

2.1 Vue d'Ensemble



2.2 Stack Technologique

Backend

- **Framework** : Flask 3.0.0 (Python)
- **Serveur Production** : Gunicorn 21.2.0
- **Architecture** : Blueprints modulaires
- **WebSocket** : Support natif Flask

Frontend

- **HTML5/CSS3** : Templates Jinja2
- **JavaScript** : ES6+ (async/await)
- **WebRTC** : Pour streaming vidéo
- **WebSocket Client** : Communication temps réel

Cloud Services

- **Azure Web App** : Hébergement
- **Azure OpenAI** : GPT-4 + Personal Voice
- **Azure Cosmos DB** : Base de données NoSQL
- **Azure Storage Blob** : Stockage fichiers
- **Azure Avatar Live API** : Streaming avatar

2.3 Flux de Données Principal



3. Composants Frontend

3.1 Interface de Configuration (4 Étapes)

Le système de configuration est divisé en 4 étapes distinctes pour une expérience utilisateur optimale.

Étape 1 : Informations de Base

Fichier : templates/avatar/avatar_step1.html

```
<form id="step1Form" action="/avatar/step1" method="POST">
  <input type="text" name="agent_name" required>
  <textarea name="description"></textarea>
  <input type="text" name="country">
  <button type="submit">Suivant</button>
</form>
```

Champs collectés :

- Nom de l'agent
- Description
- Pays (pour contexte culturel)

Validation côté client :

```
document.getElementById('step1Form').addEventListener('submit', function(e) {
  const name = document.querySelector('[name="agent_name"]').value;
  if (!name || name.length < 3) {
    e.preventDefault();
    alert('Le nom doit contenir au moins 3 caractères');
  }
});
```

```
}
});
```

Étape 2 : Configuration Avatar et Voix

Fichier : `templates/avatar/avatar_step2.html` (2,375 lignes)

Sections principales :

1. Sélection d'Avatar (pagination dynamique)

```
// Chargement dynamique depuis API Azure
async function loadAvatarImages() {
  const response = await fetch('/avatar/api/avatars');
  const data = await response.json();

  if (data.success && data.avatars) {
    allAvatars = data.avatars;
    renderAvatarPage();
  }
}

// Pagination (6 avatars par page)
function renderAvatarPage() {
  const start = (currentPage - 1) * avatarsPerPage;
  const end = start + avatarsPerPage;
  const avatarsToShow = allAvatars.slice(start, end);

  avatarsToShow.forEach((avatar, index) => {
    const card = createAvatarCard(avatar, index);
    grid.appendChild(card);
  });
}
```

2. Sélection de Style

```
function updateAvatarStyles(character) {
  const styles = avatarStyles[character];

  if (!styles || styles.length === 0) {
    styleGroup.style.display = 'none';
    styleInput.value = 'casual-sitting'; // Fallback
    return;
  }

  styleGrid.innerHTML = '';
  styles.forEach((style, index) => {
    const card = createStyleCard(style, index);
    styleGrid.appendChild(card);
  });
}
```

```
});
}
```

3. Mise à jour en Temps Réel dans Cosmos DB

```
// Sauvegarde immédiate au clic
card.addEventListener('click', async function() {
  const agentId = window.location.pathname.split('/').pop();

  const response = await fetch(
    `/avatar/api/${agentId}/update_avatar_character`,
    {
      method: 'POST',
      headers: {'Content-Type': 'application/json'},
      body: JSON.stringify({
        character: character,
        style: styleId
      })
    }
  );
});
```

4. Sélection de Voix

Trois types de voix disponibles :

- **Personal Voice** : Voix clonées personnalisées
- **Azure Neural Voices** : Voix préentraînées
- **OpenAI Voices** : Voix OpenAI

```
function selectVoice(card, voiceData) {
  // Mise à jour visuelle
  document.querySelectorAll('.voice-card').forEach(c =>
    c.classList.remove('selected')
  );
  card.classList.add('selected');

  // Sauvegarde des données
  document.getElementById('voiceType').value = voiceData.type;
  document.getElementById('voiceName').value = voiceData.name;

  if (voiceData.type === 'personal') {
    document.getElementById('customVoiceEndpoint').value =
      voiceData.endpoint;
    document.getElementById('speakerProfileId').value =
      voiceData.speakerProfile;
  }
}
```

Étape 3 : Sélection des Outils

Fichier : `templates/avatar/avatar_step3.html`

Interface de sélection multiple des 24 outils disponibles.

```
<div class="tools-grid">
  {% for tool in available_tools %}
    <div class="tool-card" onclick="toggleTool('{{ tool.name }}')">
      <input type="checkbox" name="tools[]"
        value="{{ tool.name }}" id="tool_{{ tool.name }}">
      <label for="tool_{{ tool.name }}">
        <span class="tool-icon">{{ tool.icon }}</span>
        <span class="tool-name">{{ tool.display_name }}</span>
        <p class="tool-description">{{ tool.description }}</p>
      </label>
    </div>
  {% endfor %}
</div>
```

JavaScript de gestion :

```
function toggleTool(toolName) {
  const checkbox = document.getElementById(`tool_${toolName}`);
  checkbox.checked = !checkbox.checked;

  const card = checkbox.closest('.tool-card');
  card.classList.toggle('selected', checkbox.checked);
}
```

Étape 4 : Génération du Prompt Système

Fichier : `templates/avatar/avatar_step4.html`

Fonctionnalités :

1. Génération automatique via GPT-4

```
async function generatePrompt() {
  const instructions = document.getElementById('instructions').value;

  showLoader('Génération du prompt avec GPT-4...');

  const response = await fetch(
    `/avatar/api/${agentId}/generate_prompt`,
    {
      method: 'POST',
      headers: {'Content-Type': 'application/json'},
    }
  );
}
```

```

        body: JSON.stringify({instructions: instructions})
    }
    );

    const data = await response.json();

    if (data.success) {
        document.getElementById('system_prompt').value =
            data.generated_prompt;
        hideLoader();
    }
}

```

2. Éditeur de prompt enrichi

```

<textarea id="system_prompt" name="system_prompt"
    rows="20" required>
<!-- Prompt généré automatiquement -->
</textarea>

<div class="prompt-actions">
    <button type="button" onclick="generatePrompt()">
         Générer avec IA
    </button>
    <button type="button" onclick="resetPrompt()">
         Réinitialiser
    </button>
</div>

```

3.2 Session Vocale en Temps Réel

Fichier : `templates/avatar/avatar_voice_session.html` (1,768 lignes)

3.2.1 Architecture WebRTC + WebSocket

```

// Configuration WebRTC
const peerConnection = new RTCPeerConnection({
    iceServers: iceServersFromAzure,
    bundlePolicy: 'max-bundle',
    rtcMuxPolicy: 'require'
});

// Configuration WebSocket
const socket = new WebSocket('wss://api.azure.com/avatar/live');

socket.onopen = () => {
    console.log('✅ WebSocket connecté');
    sendSessionUpdate();
};

```

3.2.2 Gestion du Flux Audio/Vidéo

Séparation des flux :

```
// Flux audio séparé (pour éviter les bugs Azure)
peerConnection.addTransceiver('audio', {direction: 'recvonly'});

// Flux vidéo séparé
peerConnection.addTransceiver('video', {direction: 'recvonly'});

// Gestion des tracks
peerConnection.ontrack = (event) => {
  const track = event.track;

  if (track.kind === 'video') {
    videoElement.srcObject = event.streams[0];
  } else if (track.kind === 'audio') {
    audioElement.srcObject = event.streams[0];
  }
};
```

3.2.3 Capture et Envoi Audio

```
async function startAudioCapture() {
  const stream = await navigator.mediaDevices.getUserMedia({
    audio: {
      echoCancellation: true,
      noiseSuppression: true,
      sampleRate: 16000
    }
  });

  const audioContext = new AudioContext();
  const source = audioContext.createMediaStreamSource(stream);
  const processor = audioContext.createScriptProcessor(4096, 1, 1);

  processor.onaudioprocess = (e) => {
    const audioData = e.inputBuffer.getChannelData(0);
    sendAudioToServer(audioData);
  };

  source.connect(processor);
  processor.connect(audioContext.destination);
}
```

3.2.4 Gestion des Messages WebSocket


```

socket.onmessage = (event) => {
  const message = JSON.parse(event.data);

  switch(message.type) {
    case 'session.created':
      handleSessionCreated(message);
      break;

    case 'session.updated':
      handleSessionUpdated(message);
      break;

    case 'response.audio.delta':
      handleAudioDelta(message);
      break;

    case 'response.done':
      handleResponseComplete(message);
      break;

    case 'error':
      handleError(message);
      break;
  }
};

```

3.3 Galerie d'Agents

Fichier : templates/avatar/avatar_gallery.html (870 lignes)

3.3.1 Affichage des Cartes

```

<div class="agents-grid">
  {% for avatar in avatars %}
    <div class="agent-card" data-status="{{ avatar.status }}">
      <div class="avatar-preview">
        <i class="bi bi-person-video3"></i>
      </div>

      <h3 class="agent-name">{{ avatar.agent_name }}</h3>
      <p class="agent-description">{{ avatar.description }}</p>

      <div class="agent-meta">
        <div class="meta-item">
          <i class="bi bi-person-badge"></i>
          <span>Avatar: {{ avatar.avatar_name }}</span>
        </div>
        <div class="meta-item">
          <i class="bi bi-mic-fill"></i>
          <span>Voix: {{ avatar.voice_name }}</span>
        </div>
      </div>
    </div>
  {% endfor %}
</div>

```

```

        </div>
    </div>

    <!-- Snippet API -->
    <div class="api-snippet">
        <div class="api-snippet-header">
            <span>API Endpoint</span>
            <button onclick="copyApiSnippet('{{ avatar.agent_id }}')">
                Copier
            </button>
        </div>
        <div class="api-snippet-body">
            <pre>POST https://www.waka.azurewebsites.net/api/agent/{{
avatar.agent_id }}/audio</pre>
        </div>
    </div>

    <div class="agent-actions">
        <a href="/avatar/call/{{ avatar.agent_id }}" class="btn-call">
            Lancer
        </a>
        <button class="btn-edit"
            onclick="editAvatar('{{ avatar.agent_id }}')">
            Modifier
        </button>
        <button class="btn-delete"
            onclick="deleteAvatar('{{ avatar.agent_id }}')">
            Supprimer
        </button>
    </div>
</div>
{% endfor %}
</div>

```

3.3.2 Filtres et Recherche

```

function filterAvatars() {
    const searchTerm = document.getElementById('searchAvatars')
        .value.toLowerCase();
    const statusFilter = document.getElementById('filterStatus').value;

    document.querySelectorAll('.agent-card').forEach(card => {
        const name = card.querySelector('.agent-name')
            .textContent.toLowerCase();
        const status = card.dataset.status;

        const matchesSearch = name.includes(searchTerm);
        const matchesStatus = !statusFilter || status === statusFilter;

        card.style.display = (matchesSearch && matchesStatus)
            ? 'block'
    });
}

```

```

        : 'none';
    });
}

```

4. Composants Backend

4.1 Blueprints Flask

4.1.1 Structure des Blueprints

Fichier : `Blueprints/avatar_routes.py` (1,200+ lignes)

```

from flask import Blueprint, render_template, request, jsonify

avatar_bp = Blueprint('avatar', __name__, url_prefix='/avatar')

```

Routes principales :

1. **Galerie** : `/avatar/gallery`
2. **Configuration Étape 1** : `/avatar/step1`
3. **Configuration Étape 2** : `/avatar/step2/<agent_id>`
4. **Configuration Étape 3** : `/avatar/step3/<agent_id>`
5. **Configuration Étape 4** : `/avatar/step4/<agent_id>`
6. **Session Vocale** : `/avatar/call/<agent_id>`
7. **API Avatars** : `/avatar/api/avatars`
8. **API Update Character** : `/avatar/api/<agent_id>/update_avatar_character`

4.1.2 Route Galerie

```

@avatar_bp.route('/gallery')
def avatar_gallery():
    """Galerie des agents avatar"""
    try:
        from configuration.cosmos_config import get_avatar_container

        container = get_avatar_container()

        # Récupérer tous les avatars
        query = "SELECT * FROM c ORDER BY c.created_at DESC"
        avatars = list(container.query_items(
            query=query,
            enable_cross_partition_query=True
        ))

        # Statistiques
        total = len(avatars)

```

```

        active_count = sum(1 for a in avatars
                            if a.get('status') == 'active')
        draft_count = total - active_count

    return render_template(
        'avatar/avatar_gallery.html',
        avatars=avatars,
        total=total,
        active_count=active_count,
        draft_count=draft_count
    )

except Exception as e:
    logger.exception("Erreur galerie avatars")
    return jsonify({'error': str(e)}), 500

```

4.1.3 Route Configuration Étape 2

```

@avatar_bp.route('/step2/<agent_id>', methods=['GET', 'POST'])
def avatar_config_step2(agent_id):
    """Configuration avatar et voix"""

    if request.method == 'GET':
        # Récupérer la config existante
        config = get_avatar_config(agent_id)

        # Charger les voix disponibles
        personal_voices = get_personal_voices()

        return render_template(
            'avatar/avatar_step2.html',
            agent_id=agent_id,
            config=config,
            personal_voices=personal_voices
        )

    else: # POST
        # Sauvegarder la configuration
        voice_data = {
            'voice_type': request.form.get('voice_type'),
            'voice_id': request.form.get('voice_id'),
            'voice_name': request.form.get('voice_name'),
            'avatar_character': request.form.get('avatar_character'),
            'avatar_style': request.form.get('avatar_style'),
            'current_step': 2
        }

        update_avatar_config(agent_id, voice_data)

        logger.info(f"✅ Step 2 terminé pour {agent_id}")

```

```
return redirect(url_for('avatar.avatar_config_step3',
                        agent_id=agent_id))
```

4.1.4 Route Mise à Jour Avatar (Temps Réel)

```
@avatar_bp.route('/api/<agent_id>/update_avatar_character',
                 methods=['POST'])
def update_avatar_character(agent_id):
    """
    Mise à jour immédiate du character et style dans Cosmos DB
    Appelé lors du clic sur un avatar ou style
    """
    try:
        from configuration.cosmos_config import update_avatar_config

        data = request.get_json()
        character = data.get('character')
        style = data.get('style')

        if not character:
            return jsonify({
                'success': False,
                'error': 'Character requis'
            }), 400

        # Mise à jour dans Cosmos DB
        update_data = {
            'avatar_character': character
        }

        if style:
            update_data['avatar_style'] = style

        update_avatar_config(agent_id, update_data)

        logger.info(f"☑ Avatar mis à jour: {agent_id} -> "
                    f"{character} ({style})")

        return jsonify({
            'success': True,
            'message': f'Avatar character mis à jour: {character}',
            'character': character,
            'style': style
        })

    except Exception as e:
        logger.exception("Erreur mise à jour avatar character")
        return jsonify({
            'success': False,
            'error': str(e)
        }), 500
```

4.1.5 Route Génération Prompt GPT-4

```
@avatar_bp.route('/api/<agent_id>/generate_prompt', methods=['POST'])
def generate_prompt_api(agent_id):
    """
    Génération automatique du prompt système via GPT-4
    """
    try:
        from openai import OpenAI

        client = OpenAI(
            api_key=os.getenv("AZURE_OPENAI_API_KEY"),
            base_url=os.getenv("AZURE_OPENAI_ENDPOINT")
        )

        # Récupérer les instructions utilisateur
        data = request.get_json()
        instructions = data.get('instructions', '')

        # Récupérer la config de l'agent
        config = get_avatar_config(agent_id)

        # Construire le prompt de génération
        generation_prompt = f"""
Génère un prompt système professionnel pour un agent IA avatar
avec les caractéristiques suivantes:

**Nom**: {config.get('agent_name')}
**Description**: {config.get('description')}
**Pays**: {config.get('country', 'Burkina Faso')}
**Outils activés**: {' '.join(config.get('selected_tools', []))}

**Instructions personnalisées**:
{instructions}

**Consignes importantes**:

1. **IDENTITÉ ET PERSONNALITÉ**
   - Présente-toi comme {config.get('agent_name')}
   - Ton rôle : {config.get('description')}
   - Sois chaleureux, professionnel et serviable

2. **UTILISATION DES OUTILS**
   - Tu as accès à ces outils : {' '.join(config.get('selected_tools', []))}
   - Utilise-les PROACTIVEMENT quand nécessaire
   - Annonce toujours avant d'utiliser un outil

3. **STYLE DE COMMUNICATION**
   - Réponses concises (2-3 phrases maximum)
   - Ton conversationnel et naturel
   - Adapte-toi au contexte culturel de {config.get('country')}
        """
```

4. ****CONTEXTE CULTUREL****

- Si un pays est mentionné dans la consigne :
 - * Lister au moins 20 expressions courantes et citations de ce pays
 - * Adopter une attitude et des références culturelles du pays
 - * Utiliser ces expressions MODÉRÉMENT et naturellement (pas à chaque phrase)
 - * Varier les expressions utilisées

5. ****FIN DE CONVERSATION****

- Si l'agent utilise l'outil "end_conversation"
- NE PAS réanimer la conversation même si l'utilisateur parle après
- La conversation est définitivement terminée

Structure le prompt en sections claires.

Fais environ 300-500 mots.

Réponds uniquement avec le prompt système, sans introduction.

"""

```
# Appel GPT-4
response = client.chat.completions.create(
    model="gpt-4",
    messages=[
        {"role": "system", "content": "Tu es un expert en "
        "création de prompts pour agents IA."},
        {"role": "user", "content": generation_prompt}
    ],
    temperature=0.7,
    max_tokens=1500
)
```

```
generated_prompt = response.choices[0].message.content
```

```
# Ajouter la section outils automatiquement
from tools import get_tools_definition
```

```
all_tools = get_tools_definition()
```

```
tools_section = """
```

🛠️ OUTILS DISPONIBLES - DÉFINITIONS COMPLÈTES

IMPORTANT: Ces outils sont déjà configurés. Utilise-les ACTIVEMENT!

📋 RÈGLES IMPÉRATIVES

1. ☒ ****Annonce**** avant d'appeler
2. ☒ ****Appelle IMMÉDIATEMENT**** dès que tu as les infos
3. ☒ ****Réponds avec le résultat**** dès réception
4. ☒ ****NE propose JAMAIS**** - AGIS directement!

```

"""

# Ajouter les définitions complètes des outils
for tool in all_tools:
    tool_name = tool.get("name", "")
    tool_desc = tool.get("description", "")

    tools_section += f"""
### {tool_name}
{tool_desc}

"""

# Combiner le prompt généré + section outils
final_prompt = generated_prompt + tools_section

return jsonify({
    'success': True,
    'generated_prompt': final_prompt
})

except Exception as e:
    logger.exception("Erreur génération prompt")
    return jsonify({
        'success': False,
        'error': str(e)
    }), 500

```

4.2 Configuration Azure Voice Live

Fichier : `configuration/voice_live_config.py` (400+ lignes)

4.2.1 Classe VoiceLiveClient

```

class VoiceLiveClient:
    """
    Client pour Azure AI Avatar Live API
    Gère la connexion WebSocket et la session RTCPeerConnection
    """

    def __init__(self, endpoint, api_key):
        self.endpoint = endpoint
        self.api_key = api_key
        self.websocket = None
        self.session_id = None

    async def connect(self):
        """Établit la connexion WebSocket"""
        headers = {
            'api-key': self.api_key,

```



```
        'Content-Type': 'application/json'
    }

    self.websocket = await websockets.connect(
        self.endpoint,
        extra_headers=headers,
        ping_interval=30,
        ping_timeout=10
    )

    logger.info("☑ Connexion WebSocket établie")

async def create_session(self, config):
    """
    Crée une session avatar

    Args:
        config: Configuration avatar (character, style, voice, etc.)
    """
    message = {
        "type": "session.update",
        "session": {
            "turn_detection": {
                "type": "server_vad",
                "threshold": 0.5,
                "prefix_padding_ms": 300,
                "silence_duration_ms": 500
            },
            "avatar": {
                "enabled": True,
                "character": config.get('character', 'lisa'),
                "style": config.get('style', 'casual-sitting'),
                "crop": {
                    "topLeft": config.get('crop_top_left',
                                          [0, 0]),
                    "bottomRight": config.get('crop_bottom_right',
                                              [1920, 1080])
                }
            },
            "voice": config.get('voice_id'),
            "instructions": config.get('system_prompt'),
            "tools": config.get('tools_definitions', []),
            "temperature": 0.8,
            "max_response_output_tokens": 1500
        }
    }

    await self.websocket.send(json.dumps(message))

    # Attendre la confirmation
    response = await self.websocket.recv()
    data = json.loads(response)

    if data.get('type') == 'session.created':
```

```

        self.session_id = data.get('session', {}).get('id')
        logger.info(f"✅ Session créée: {self.session_id}")

    return data

    async def send_audio(self, audio_data):
        """Envoie des données audio au serveur"""
        message = {
            "type": "input_audio_buffer.append",
            "audio": audio_data # Base64 encoded
        }

        await self.websocket.send(json.dumps(message))

    async def commit_audio(self):
        """Finalise l'envoi audio et déclenche la réponse"""
        message = {
            "type": "input_audio_buffer.commit"
        }

        await self.websocket.send(json.dumps(message))

    async def receive_messages(self, callback):
        """
        Reçoit les messages du serveur en continu

        Args:
            callback: Fonction appelée pour chaque message reçu
        """
        try:
            async for message in self.websocket:
                data = json.dumps(message)
                await callback(data)

        except websockets.exceptions.ConnectionClosed:
            logger.warning("Connexion WebSocket fermée")

    async def close(self):
        """Ferme la connexion WebSocket"""
        if self.websocket:
            await self.websocket.close()
            logger.info("WebSocket fermé")

```

4.3 Gestion Cosmos DB

Fichier : `configuration/cosmos_config.py` (600+ lignes)

4.3.1 Configuration Cosmos DB

```

from azure.cosmos import CosmosClient, exceptions
import os

```

```
# Configuration
COSMOS_ENDPOINT = os.getenv("COSMOS_DB_ENDPOINT")
COSMOS_KEY = os.getenv("COSMOS_DB_KEY")
DATABASE_NAME = "WakaVoiceDB"
CONTAINER_NAME = "Avatars"

def get_cosmos_client():
    """Retourne le client Cosmos DB"""
    return CosmosClient(COSMOS_ENDPOINT, COSMOS_KEY)

def get_avatar_container():
    """Retourne le container des avatars"""
    client = get_cosmos_client()
    database = client.get_database_client(DATABASE_NAME)
    container = database.get_container_client(CONTAINER_NAME)
    return container
```

4.3.2 Opérations CRUD

```
def create_avatar_config(agent_id, config_data):
    """
    Crée une nouvelle configuration d'avatar

    Args:
        agent_id: ID unique de l'agent
        config_data: Dictionnaire avec la configuration
    """
    try:
        container = get_avatar_container()

        document = {
            'id': agent_id,
            'agent_id': agent_id,
            'agent_name': config_data.get('agent_name'),
            'description': config_data.get('description'),
            'country': config_data.get('country'),
            'status': 'draft',
            'current_step': 1,
            'created_at': datetime.utcnow().isoformat(),
            'updated_at': datetime.utcnow().isoformat()
        }

        container.create_item(body=document)

        logger.info(f"✅ Avatar créé: {agent_id}")

        return document

    except exceptions.CosmosResourceExistsError:
        logger.error(f"❌ Avatar {agent_id} existe déjà")
```

```
        raise
    except Exception as e:
        logger.exception("Erreur création avatar")
        raise

def get_avatar_config(agent_id):
    """
    Récupère la configuration d'un avatar

    Args:
        agent_id: ID de l'agent

    Returns:
        dict: Configuration de l'avatar ou None
    """
    try:
        container = get_avatar_container()

        item = container.read_item(
            item=agent_id,
            partition_key=agent_id
        )

        return item

    except exceptions.CosmosResourceNotFoundError:
        logger.warning(f"⚠ Avatar {agent_id} introuvable")
        return None
    except Exception as e:
        logger.exception("Erreur lecture avatar")
        raise

def update_avatar_config(agent_id, update_data):
    """
    Met à jour la configuration d'un avatar

    Args:
        agent_id: ID de l'agent
        update_data: Dictionnaire avec les données à mettre à jour
    """
    try:
        container = get_avatar_container()

        # Récupérer le document existant
        existing_item = container.read_item(
            item=agent_id,
            partition_key=agent_id
        )

        # Fusionner les données
        existing_item.update(update_data)
        existing_item['updated_at'] = datetime.utcnow().isoformat()

        # Sauvegarder
```

```

        container.replace_item(
            item=agent_id,
            body=existing_item
        )

        logger.info(f"✅ Avatar mis à jour: {agent_id}")

        return existing_item

    except exceptions.CosmosResourceNotFoundError:
        logger.error(f"❌ Avatar {agent_id} introuvable")
        raise
    except Exception as e:
        logger.exception("Erreur mise à jour avatar")
        raise

def delete_avatar_config(agent_id):
    """
    Supprime la configuration d'un avatar

    Args:
        agent_id: ID de l'agent
    """
    try:
        container = get_avatar_container()

        container.delete_item(
            item=agent_id,
            partition_key=agent_id
        )

        logger.info(f"✅ Avatar supprimé: {agent_id}")

    except exceptions.CosmosResourceNotFoundError:
        logger.warning(f"⚠️ Avatar {agent_id} introuvable")
    except Exception as e:
        logger.exception("Erreur suppression avatar")
        raise

def list_avatar_configs(status=None):
    """
    Liste les configurations d'avatars avec filtre optionnel

    Args:
        status: Filtre par status (draft, active, etc.)

    Returns:
        list: Liste des configurations
    """
    try:
        container = get_avatar_container()

        if status:
            query = f"SELECT * FROM c WHERE c.status = @status " \

```

```

        f"ORDER BY c.created_at DESC"
        parameters = [{"name": "@status", "value": status}]
    else:
        query = "SELECT * FROM c ORDER BY c.created_at DESC"
        parameters = []

    items = list(container.query_items(
        query=query,
        parameters=parameters,
        enable_cross_partition_query=True
    ))

    logger.info(f"☑ {len(items)} avatars trouvés")

    return items

except Exception as e:
    logger.exception("Erreur listage avatars")
    raise

```

5. Intégration Azure Services

5.1 Azure OpenAI Integration

5.1.1 Configuration

```

from openai import AzureOpenAI
import os

client = AzureOpenAI(
    api_key=os.getenv("AZURE_OPENAI_API_KEY"),
    api_version="2024-08-01-preview",
    azure_endpoint=os.getenv("AZURE_OPENAI_ENDPOINT")
)

DEPLOYMENT_NAME = os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME",
                             "gpt-4-turbo")

```

5.1.2 Appels API

```

def call_gpt4(messages, tools=None, temperature=0.8):
    """
    Appelle GPT-4 avec support des tools

    Args:
        messages: Liste des messages de conversation
        tools: Liste des définitions d'outils (optionnel)
    """

```

```

        temperature: Température de génération

Returns:
    dict: Réponse de GPT-4
"""
try:
    params = {
        "model": DEPLOYMENT_NAME,
        "messages": messages,
        "temperature": temperature,
        "max_tokens": 1500
    }

    if tools:
        params["tools"] = tools
        params["tool_choice"] = "auto"

    response = client.chat.completions.create(**params)

    return response

except Exception as e:
    logger.exception("Erreur appel GPT-4")
    raise

```

5.2 Azure Personal Voice

5.2.1 Gestion des Voix Personnalisées

Fichier : `configuration/personal_voice_storage.py`

```

def list_personal_voices():
    """
    Liste les voix personnalisées disponibles

    Returns:
        list: Liste des voix avec métadonnées
    """
    try:
        # Appel à l'API Azure Speech
        endpoint = os.getenv("AZURE_SPEECH_ENDPOINT")
        api_key = os.getenv("AZURE_SPEECH_KEY")

        url = f"{endpoint}/voice-synthesis/voices/personal"

        headers = {
            'Ocp-Apim-Subscription-Key': api_key,
            'Content-Type': 'application/json'
        }

        response = requests.get(url, headers=headers)
    
```

```
        response.raise_for_status()

        voices = response.json()

        logger.info(f"✅ {len(voices)} voix personnalisées trouvées")

        return voices

    except Exception as e:
        logger.exception("Erreur listage voix personnalisées")
        return []

def create_personal_voice(name, audio_samples):
    """
    Crée une nouvelle voix personnalisée

    Args:
        name: Nom de la voix
        audio_samples: Liste de fichiers audio pour le clonage

    Returns:
        dict: Informations sur la voix créée
    """
    try:
        endpoint = os.getenv("AZURE_SPEECH_ENDPOINT")
        api_key = os.getenv("AZURE_SPEECH_KEY")

        url = f"{endpoint}/voice-synthesis/voices/personal"

        headers = {
            'Ocp-Apim-Subscription-Key': api_key
        }

        # Préparer les fichiers
        files = [
            ('audioSamples', (f'sample{i}.wav', open(sample, 'rb'),
                              'audio/wav'))
            for i, sample in enumerate(audio_samples)
        ]

        data = {
            'name': name,
            'locale': 'fr-FR',
            'description': f'Voix personnalisée: {name}'
        }

        response = requests.post(
            url,
            headers=headers,
            files=files,
            data=data
        )
        response.raise_for_status()
```



```
voice_info = response.json()

logger.info(f"✅ Voix personnalisée créée: {name}")

return voice_info

except Exception as e:
    logger.exception("Erreur création voix personnalisée")
    raise
```

5.3 Azure Storage Blob

5.3.1 Upload de Fichiers

```
from azure.storage.blob import BlobServiceClient
import os

def upload_audio_file(file_path, blob_name):
    """
    Upload un fichier audio vers Azure Blob Storage

    Args:
        file_path: Chemin du fichier local
        blob_name: Nom du blob dans le container

    Returns:
        str: URL du blob uploadé
    """
    try:
        connection_string = os.getenv("AZURE_STORAGE_CONNECTION_STRING")
        container_name = "audio-samples"

        blob_service_client = BlobServiceClient\
            .from_connection_string(connection_string)

        container_client = blob_service_client\
            .get_container_client(container_name)

        # Créer le container s'il n'existe pas
        try:
            container_client.create_container()
        except:
            pass # Container existe déjà

        # Upload le fichier
        with open(file_path, "rb") as data:
            blob_client = container_client.get_blob_client(blob_name)
            blob_client.upload_blob(data, overwrite=True)

        blob_url = blob_client.url
```

```
        logger.info(f"✅ Fichier uploadé: {blob_name}")

        return blob_url

    except Exception as e:
        logger.exception("Erreur upload fichier")
        raise
```

6. Système de Configuration

6.1 Workflow de Configuration en 4 Étapes

6.1.1 Étape 1 : Informations de Base

Données collectées :

- Nom de l'agent
- Description
- Pays (pour contexte culturel)

Validation backend :

```
@avatar_bp.route('/step1', methods=['POST'])
def create_avatar_step1():
    agent_name = request.form.get('agent_name', '').strip()
    description = request.form.get('description', '').strip()
    country = request.form.get('country', 'Burkina Faso').strip()

    # Validation
    if not agent_name or len(agent_name) < 3:
        return jsonify({
            'success': False,
            'error': 'Le nom doit contenir au moins 3 caractères'
        }), 400

    # Génération d'un ID unique
    agent_id = str(uuid.uuid4())

    # Création dans Cosmos DB
    config_data = {
        'agent_name': agent_name,
        'description': description,
        'country': country,
        'status': 'draft',
        'current_step': 1
    }

    create_avatar_config(agent_id, config_data)

    # Redirection vers l'étape 2
```

```
return redirect(url_for('avatar.avatar_config_step2',
                        agent_id=agent_id))
```

6.1.2 Étape 2 : Avatar et Voix

Données collectées :

- Character d'avatar (lisa, harry, jeff, lori, max, meg)
- Style d'avatar (casual-sitting, formal-standing, etc.)
- Type de voix (personal, azure, openai)
- Configuration voix spécifique

Particularité : Mise à jour en temps réel

Contrairement aux autres étapes, l'étape 2 permet des mises à jour immédiates dans Cosmos DB lors de la sélection :

```
// Clic sur avatar → Appel API immédiat
card.addEventListener('click', async function() {
  const response = await fetch(
    `/avatar/api/${agentId}/update_avatar_character`,
    {
      method: 'POST',
      headers: {'Content-Type': 'application/json'},
      body: JSON.stringify({
        character: character,
        style: styleId
      })
    }
  );
});
```

Avantage : Évite la perte de données en cas de changement de page.

6.1.3 Étape 3 : Sélection des Outils

24 outils disponibles :

1. search_web - Recherche web
2. send_email - Envoi email
3. get_weather_forecast - Météo
4. create_cv - Création CV
5. convert_currency - Conversion devise
6. search_flights - Recherche vols
7. search_hotels - Recherche hôtels
8. book_flight - Réservation vol
9. book_hotel - Réservation hôtel
10. search_exercises - Exercices fitness

11. search_dog_breeds - Races de chiens
12. search_knowledge_base - Base de connaissances
13. get_health_advice - Conseils santé
14. get_news - Actualités
15. search_places - Recherche lieux
16. translate_text - Traduction
17. calculate - Calculs
18. end_conversation - Fin conversation
19. get_prayer_times - Heures de prière
20. find_pharmacy - Pharmacie
21. estimate_taxi_fare - Estimation taxi
22. get_bus_schedule - Horaires bus
23. get_school_info - Infos scolaires
24. get_government_service_info - Services gouvernementaux
25. calculate_tax - Calcul d'impôts

Backend de sauvegarde :

```
@avatar_bp.route('/step3/<agent_id>', methods=['POST'])
def avatar_config_step3(agent_id):
    selected_tools = request.form.getlist('tools[]')

    # Validation : au moins 1 outil
    if not selected_tools:
        return jsonify({
            'success': False,
            'error': 'Sélectionnez au moins un outil'
        }), 400

    # Sauvegarde
    update_data = {
        'selected_tools': selected_tools,
        'current_step': 3
    }

    update_avatar_config(agent_id, update_data)

    return redirect(url_for('avatar.avatar_config_step4',
                            agent_id=agent_id))
```

6.1.4 Étape 4 : Génération du Prompt

Deux modes de saisie :

1. Génération automatique via GPT-4

- L'utilisateur fournit des instructions courtes
- GPT-4 génère un prompt complet et structuré
- Injection automatique de la section outils

2. Saisie manuelle

- L'utilisateur écrit directement le prompt
- Possibilité de modifier le prompt généré

Backend :

```
@avatar_bp.route('/step4/<agent_id>', methods=['POST'])
def avatar_config_step4(agent_id):
    system_prompt = request.form.get('system_prompt', '')

    if not system_prompt or len(system_prompt) < 50:
        return jsonify({
            'success': False,
            'error': 'Le prompt doit contenir au moins 50 caractères'
        }), 400

    # Marquer comme terminé
    update_data = {
        'system_prompt': system_prompt,
        'instructions': system_prompt,
        'current_step': 4,
        'status': 'completed'
    }

    update_avatar_config(agent_id, update_data)

    logger.info(f"✅ Configuration terminée pour {agent_id}")

    # Redirection vers la galerie
    return redirect(url_for('avatar.avatar_gallery'))
```

6.2 Gestion des Erreurs et Validation

6.2.1 Validation Frontend

```
// Validation avant soumission
function validateStep2Form() {
    const avatarChar = document.getElementById('avatarCharacter').value;
    const voiceType = document.getElementById('voiceType').value;

    if (!avatarChar) {
        alert('⚠️ Veuillez sélectionner un avatar');
        return false;
    }

    if (!voiceType) {
        alert('⚠️ Veuillez sélectionner une voix');
        return false;
    }
}
```

```

    return true;
}

```

6.2.2 Gestion des Erreurs Backend

```

@avatar_bp.errorhandler(Exception)
def handle_error(error):
    """Gestionnaire d'erreurs global pour le blueprint avatar"""

    logger.exception("Erreur non gérée")

    # En production, ne pas exposer les détails
    if os.getenv('FLASK_ENV') == 'production':
        return jsonify({
            'success': False,
            'error': 'Une erreur est survenue. Veuillez réessayer.'
        }), 500
    else:
        return jsonify({
            'success': False,
            'error': str(error),
            'type': type(error).__name__
        }), 500

```

7. Gestion des Outils (24 Tools)

7.1 Architecture des Outils

Chaque outil suit le même pattern :

```

# tools/tool_<nom>.py

def get_tool_definition():
    """Retourne la définition OpenAI du tool"""
    return {
        "type": "function",
        "name": "nom_de_l_outil",
        "description": "Description détaillée...",
        "parameters": {
            "type": "object",
            "properties": {
                "param1": {
                    "type": "string",
                    "description": "Description param1"
                }
            }
        },
        "required": ["param1"]
    }

```

```
    }  
}  
  
def execute(arguments):  
    """Exécute l'outil avec les arguments fournis"""  
    # Logique métier  
    result = do_something(arguments)  
    return result
```

7.2 Orchestrateur d'Outils

Fichier : `tools/__init__.py`

```
from . import tool_search_web  
from . import tool_email  
from . import tool_weather  
# ... imports des 24 outils  
  
def get_tools_definition():  
    """  
    Retourne toutes les définitions d'outils  
  
    Returns:  
        list: Liste des définitions au format OpenAI  
    """  
    tools = [  
        tool_search_web.get_tool_definition(),  
        tool_email.get_tool_definition(),  
        tool_weather.get_tool_definition(),  
        # ... tous les outils  
    ]  
  
    return tools  
  
def execute_tool(tool_name, arguments):  
    """  
    Exécute un outil spécifique  
  
    Args:  
        tool_name: Nom de l'outil  
        arguments: Arguments pour l'outil  
  
    Returns:  
        dict: Résultat de l'exécution  
    """  
    tool_map = {  
        "search_web": tool_search_web,  
        "send_email": tool_email,  
        "get_weather_forecast": tool_weather,  
        # ... mapping complet  
    }
```

```

tool_module = tool_map.get(tool_name)

if not tool_module:
    return {
        "status": "error",
        "message": f"Outil '{tool_name}' non trouvé"
    }

try:
    result = tool_module.execute(arguments)
    return result
except Exception as e:
    return {
        "status": "error",
        "message": f"Erreur: {str(e)}"
    }

```

7.3 Exemples d'Outils Détaillés

7.3.1 Outil Météo

Fichier : `tools/tool_weather.py` (231 lignes)

```

import os
import requests
from dotenv import load_dotenv

load_dotenv()

WEATHER_API_KEY = os.getenv("WEATHER_API_KEY", "")
DEFAULT_COUNTRY = "Burkina Faso"

def get_tool_definition():
    return {
        "type": "function",
        "name": "get_weather_forecast",
        "description": """"OUTIL PRIORITAIRE pour la MÉTÉO -
        Prévisions météo détaillées pour toute ville.

        UTILISATION OBLIGATOIRE pour:
        - Météo, temps qu'il fait, prévisions
        - Température, pluie, vent, humidité
        - Climat, chaleur, froid, ensoleillé

        NE JAMAIS utiliser search_web pour la météo.""",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {
                    "type": "string",

```



```

        "description": """"Nom de la VILLE pour les
        prévisions météo.

        EXEMPLES VALIDES:
        - Ouagadougou (capitale Burkina Faso)
        - Bobo-Dioulasso
        - Paris (France)

        FORMAT: Nom complet de la ville""""
    },
    "country": {
        "type": "string",
        "description": """"Nom du PAYS (optionnel,
        par défaut 'Burkina Faso')""",
        "default": "Burkina Faso"
    },
    "days": {
        "type": "integer",
        "description": """"Nombre de JOURS de prévisions
        (entre 1 et 5).

        CONVERSION depuis langage naturel:
        - "météo aujourd'hui" → 1
        - "prévisions pour demain" → 2
        - "météo de la semaine" → 5""",
        "default": 3
    }
},
"required": ["city"]
}
}

```

```

def get_weather_forecast(city, country=DEFAULT_COUNTRY, days=3):
    """
    Obtient les prévisions météo via WeatherAPI

    Args:
        city: Nom de la ville
        country: Pays (optionnel)
        days: Nombre de jours (1-5)

    Returns:
        dict: Résultat avec météo actuelle et prévisions
    """
    if not WEATHER_API_KEY:
        return {
            "status": "error",
            "message": "WEATHER_API_KEY non configuré"
        }

    try:
        # Limiter le nombre de jours
        days = min(max(1, days), 5)
    
```

```
# WeatherAPI.com - Forecast
endpoint = "http://api.weatherapi.com/v1/forecast.json"
params = {
    "key": WEATHER_API_KEY,
    "q": f"{city},{country}",
    "days": days,
    "lang": "fr",
    "aqi": "no",
    "alerts": "no"
}

response = requests.get(endpoint, params=params, timeout=10)

if response.status_code != 200:
    return {
        "status": "error",
        "message": f"Erreur API: {response.status_code}"
    }

data = response.json()

# Météo actuelle
current = {
    "temperature": data["current"]["temp_c"],
    "feels_like": data["current"]["feelslike_c"],
    "condition": data["current"]["condition"]["text"],
    "humidity": data["current"]["humidity"],
    "wind_kph": data["current"]["wind_kph"],
    "precipitation_mm": data["current"]["precip_mm"],
    "uv": data["current"]["uv"]
}

# Prévisions
forecasts = []
for day in data["forecast"]["forecastday"]:
    forecasts.append({
        "date": day["date"],
        "max_temp": day["day"]["maxtemp_c"],
        "min_temp": day["day"]["mintemp_c"],
        "condition": day["day"]["condition"]["text"],
        "chance_of_rain": day["day"]["daily_chance_of_rain"],
        "sunrise": day["astro"]["sunrise"],
        "sunset": day["astro"]["sunset"]
    })

return {
    "status": "success",
    "type": "forecast",
    "city": data["location"]["name"],
    "country": data["location"]["country"],
    "current": current,
    "forecasts": forecasts
}
```

```

except requests.exceptions.Timeout:
    return {
        "status": "error",
        "message": "Timeout API météo"
    }
except Exception as e:
    return {
        "status": "error",
        "message": f"Erreur: {str(e)}"
    }

def execute(arguments):
    """Point d'entrée pour l'exécution"""
    city = arguments.get("city", "")
    country = arguments.get("country", DEFAULT_COUNTRY)
    days = arguments.get("days", 3)

    return get_weather_forecast(city, country, days)

```

7.3.2 Outil Email

Fichier : `tools/tool_email.py`

```

from azure.communication.email import EmailClient
import os

def get_tool_definition():
    return {
        "type": "function",
        "name": "send_email",
        "description": """Envoie un email professionnel.

        UTILISATION:
        - Envoyer email, mail, courriel
        - Communication professionnelle
        - Notification importante""",
        "parameters": {
            "type": "object",
            "properties": {
                "to": {
                    "type": "string",
                    "description": "Email du destinataire"
                },
                "subject": {
                    "type": "string",
                    "description": "Sujet de l'email"
                },
                "body": {
                    "type": "string",
                    "description": "Corps du message"
                }
            }
        }
    }

```

```

        },
        "required": ["to", "subject", "body"]
    }
}

def send_email(to, subject, body):
    """
    Envoie un email via Azure Communication Services

    Args:
        to: Email destinataire
        subject: Sujet
        body: Corps du message

    Returns:
        dict: Résultat de l'envoi
    """
    try:
        connection_string = os.getenv(
            "AZURE_COMMUNICATION_CONNECTION_STRING"
        )
        sender_address = os.getenv("AZURE_EMAIL_SENDER")

        client = EmailClient.from_connection_string(
            connection_string
        )

        message = {
            "senderAddress": sender_address,
            "recipients": {
                "to": [{"address": to}]
            },
            "content": {
                "subject": subject,
                "plainText": body
            }
        }

        poller = client.begin_send(message)
        result = poller.result()

        return {
            "status": "success",
            "message": f"Email envoyé à {to}",
            "message_id": result['id']
        }

    except Exception as e:
        return {
            "status": "error",
            "message": f"Erreur envoi email: {str(e)}"
        }

def execute(arguments):

```

```

to = arguments.get("to")
subject = arguments.get("subject")
body = arguments.get("body")

return send_email(to, subject, body)

```

7.4 Injection des Outils dans le Prompt

Lors de la génération du prompt système, les définitions complètes des outils sont automatiquement ajoutées :

```

def append_tools_to_prompt(base_prompt, selected_tools):
    """
    Ajoute la documentation des outils au prompt

    Args:
        base_prompt: Prompt système de base
        selected_tools: Liste des noms d'outils sélectionnés

    Returns:
        str: Prompt enrichi avec documentation outils
    """
    from tools import get_tools_definition

    all_tools = get_tools_definition()

    tools_section = """

## 🛠️ OUTILS DISPONIBLES - DÉFINITIONS COMPLÈTES

IMPORTANT: Ces outils sont configurés. Utilise-les ACTIVEMENT!

### 📋 RÈGLES IMPÉRATIVES

1. ✅ Annonce avant d'appeler: "Je vérifie...", "J'envoie..."
2. ✅ Appelle IMMÉDIATEMENT dès que tu as les infos nécessaires
3. ✅ Réponds avec le résultat dès réception
4. ❌ NE propose JAMAIS - AGIS directement!

---

"""

    # Filtrer les outils sélectionnés
    selected_tool_defs = [
        tool for tool in all_tools
        if tool['name'] in selected_tools
    ]

    # Ajouter les définitions
    for tool in selected_tool_defs:

```

```

        tools_section += f"""
### 🔑 {tool['name']}

**Description:**
{tool['description']}

**Paramètres:**
```json
{json.dumps(tool['parameters'], indent=2)}

```

### Exemple d'appel:

```

{{
 "type": "function",
 "name": "{tool['name']}",
 "arguments": {{
 // Arguments ici
 }}
}}
```

---

```

"""

```

```

Ajouter exemples concrets
tools_section += """

```

## 💡 EXEMPLES D'UTILISATION

**Exemple 1 - Météo:** Utilisateur: "Quel temps fait-il à Ouagadougou?" Action: Appeler immédiatement `get_weather_forecast`

```

{
 "name": "get_weather_forecast",
 "arguments": {
 "city": "Ouagadougou",
 "country": "Burkina Faso",
 "days": 1
 }
}
```

**Exemple 2 - Email:** Utilisateur: "Envoie un email de rappel à client@example.com" Action: Appeler immédiatement `send_email`

```

{
 "name": "send_email",

```

```

 "arguments": {
 "to": "client@example.com",
 "subject": "Rappel",
 "body": "Ceci est un rappel..."
 }
 }
}

```

```

.....

```

```

return base_prompt + tools_section

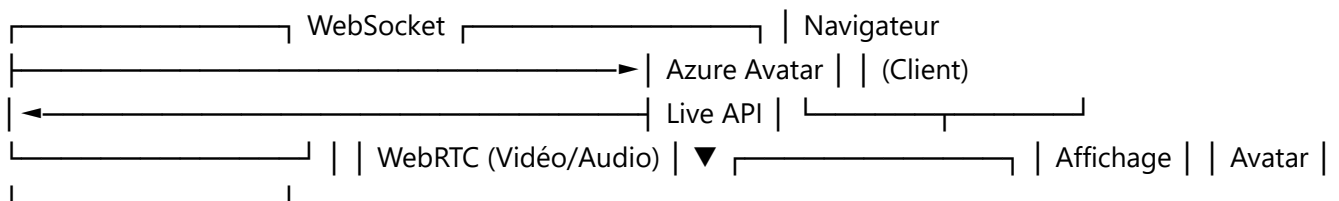
```

```

```

## ## 8. WebSocket et Temps Réel

### ### 8.1 Architecture de Communication



### ### 8.2 Établissement de la Connexion

```

```javascript
// Connexion WebSocket
const socket = new WebSocket(
  'wss://api.azure.com/avatar/live/v1',
  {
    headers: {
      'api-key': apiKey,
      'Content-Type': 'application/json'
    }
  }
);

socket.onopen = () => {
  console.log('☑️ WebSocket connecté');

  // Envoyer la configuration de session
  const sessionConfig = {
    "type": "session.update",

```

```

    "session": {
      "turn_detection": {
        "type": "server_vad",
        "threshold": 0.5,
        "prefix_padding_ms": 300,
        "silence_duration_ms": 500
      },
      "avatar": {
        "enabled": true,
        "character": "lisa",
        "style": "casual-sitting",
        "crop": {
          "topLeft": [560, 0],
          "bottomRight": [1360, 1080]
        }
      },
      "voice": voiceId,
      "instructions": systemPrompt,
      "tools": toolsDefinitions,
      "temperature": 0.8,
      "max_response_output_tokens": 1500
    }
  };

  socket.send(JSON.stringify(sessionConfig));
};

```

8.3 Gestion des Messages

```

socket.onmessage = (event) => {
  const message = JSON.parse(event.data);

  console.log('✉ Message reçu:', message.type);

  switch(message.type) {
    case 'session.created':
      console.log('✅ Session créée:', message.session.id);
      sessionId = message.session.id;
      setupWebRTC(message.session);
      break;

    case 'session.updated':
      console.log('✅ Session mise à jour');
      iceServers = message.session.ice_servers;
      initializeRTC();
      break;

    case 'response.audio_transcript.delta':
      // Transcription en temps réel
      updateTranscript(message.delta);
      break;
  }
};

```



```
    case 'response.audio.delta':
      // Audio de réponse
      playAudioChunk(message.delta);
      break;

    case 'response.function_call_arguments.delta':
      // Appel de fonction en cours
      updateFunctionCall(message);
      break;

    case 'response.function_call_arguments.done':
      // Appel de fonction terminé
      executeTool(message.call_id, message.name,
                  message.arguments);
      break;

    case 'response.done':
      console.log('✅ Réponse terminée');
      resetUI();
      break;

    case 'error':
      console.error('❌ Erreur:', message.error);
      handleError(message.error);
      break;
  }
};
```

8.4 Configuration WebRTC

```
function initializeRTC() {
  // Configuration avec ICE servers Azure
  const config = {
    iceServers: iceServers,
    bundlePolicy: 'max-bundle',
    rtcpMuxPolicy: 'require'
  };

  peerConnection = new RTCPeerConnection(config);

  // Flux audio séparé (BUG FIX: Azure Avatar)
  peerConnection.addTransceiver('audio', {
    direction: 'recvonly'
  });

  // Flux vidéo séparé
  peerConnection.addTransceiver('video', {
    direction: 'recvonly'
  });
}
```

```

// Gestion des tracks
peerConnection.ontrack = (event) => {
  console.log('📺 Track reçu:', event.track.kind);

  if (event.track.kind === 'video') {
    const videoElement = document.getElementById('avatarVideo');
    videoElement.srcObject = event.streams[0];
    videoElement.play();
  } else if (event.track.kind === 'audio') {
    const audioElement = document.getElementById('avatarAudio');
    audioElement.srcObject = event.streams[0];
    audioElement.play();
  }
};

// Gestion ICE
peerConnection.onicecandidate = (event) => {
  if (event.candidate) {
    console.log('📦 ICE candidate:', event.candidate);
  }
};

// Créer l'offre SDP
createSDPOffer();
}

async function createSDPOffer() {
  try {
    const offer = await peerConnection.createOffer();
    await peerConnection.setLocalDescription(offer);

    // Envoyer l'offre au serveur
    const message = {
      "type": "rtc.sdp",
      "sdp": offer.sdp
    };

    socket.send(JSON.stringify(message));

    console.log('📄 Offre SDP envoyée');
  } catch (error) {
    console.error('❌ Erreur création offre SDP:', error);
  }
}

```

8.5 Capture et Envoi Audio

```

let audioContext;
let mediaStreamSource;
let scriptProcessor;

```

```
async function startAudioCapture() {
  try {
    // Demander accès micro
    const stream = await navigator.mediaDevices.getUserMedia({
      audio: {
        echoCancellation: true,
        noiseSuppression: true,
        autoGainControl: true,
        sampleRate: 16000,
        channelCount: 1
      }
    });

    // Créer le contexte audio
    audioContext = new (window.AudioContext ||
      window.webkitAudioContext)();

    mediaStreamSource = audioContext.createMediaStreamSource(stream);

    // Processor pour capturer les données
    scriptProcessor = audioContext.createScriptProcessor(
      4096, 1, 1
    );

    scriptProcessor.onaudioprocess = (audioProcessingEvent) => {
      const inputBuffer = audioProcessingEvent.inputBuffer;
      const inputData = inputBuffer.getChannelData(0);

      // Convertir en base64
      const int16Array = convertFloat32ToInt16(inputData);
      const base64Audio = arrayBufferToBase64(int16Array.buffer);

      // Envoyer au serveur
      sendAudioChunk(base64Audio);
    };

    // Connecter les nodes
    mediaStreamSource.connect(scriptProcessor);
    scriptProcessor.connect(audioContext.destination);

    console.log('🎤 Capture audio démarrée');
  } catch (error) {
    console.error('❌ Erreur capture audio:', error);
  }
}

function convertFloat32ToInt16(float32Array) {
  const int16Array = new Int16Array(float32Array.length);

  for (let i = 0; i < float32Array.length; i++) {
    const s = Math.max(-1, Math.min(1, float32Array[i]));
    int16Array[i] = s < 0 ? s * 0x8000 : s * 0x7FFF;
  }
}
```

```
    }

    return int16Array;
}

function sendAudioChunk(base64Audio) {
    const message = {
        "type": "input_audio_buffer.append",
        "audio": base64Audio
    };

    socket.send(JSON.stringify(message));
}

function stopAudioCapture() {
    if (scriptProcessor) {
        scriptProcessor.disconnect();
    }

    if (mediaStreamSource) {
        mediaStreamSource.disconnect();
    }

    if (audioContext) {
        audioContext.close();
    }

    // Finaliser l'envoi audio
    const message = {
        "type": "input_audio_buffer.commit"
    };

    socket.send(JSON.stringify(message));

    console.log('🔊 Capture audio arrêtée');
}
```

9. Base de Données Cosmos DB

9.1 Modèle de Données Avatar

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "agent_id": "550e8400-e29b-41d4-a716-446655440000",
  "agent_name": "Assistant Burkina",
  "description": "Agent pour assistance générale",
  "country": "Burkina Faso",

  "avatar_character": "lisa",
  "avatar_style": "casual-sitting",
}
```

```
"avatar_customized": false,
"avatar_name": "Lisa",

"voice_type": "personal",
"voice_id": "voice-123",
"voice_name": "Voix Amadou",
"custom_voice_endpoint": "https://...",
"speaker_profile_id": "profile-456",
"voice_locale": "fr-FR",
"voice_gender": "Male",
"speaking_rate": 1.0,
"enable_output_timestamps": true,

"selected_tools": [
  "get_weather_forecast",
  "send_email",
  "search_web",
  "translate_text"
],

"system_prompt": "Tu es Assistant Burkina, un agent IA...",
"instructions": "Tu es Assistant Burkina, un agent IA...",

"model_id": "gpt-4-turbo",
"model_name": "GPT-4 Turbo",
"gpt_model": "gpt-4-turbo",

"status": "completed",
"current_step": 4,

"created_at": "2025-11-19T10:30:00.000Z",
"updated_at": "2025-11-19T11:45:00.000Z",

"phone_number": "+22670123456",
"language": "fr-FR"
}
```

9.2 Partitionnement

Clé de partition : `/agent_id`

Justification :

- Accès direct par ID d'agent (très fréquent)
- Distribution équilibrée des données
- Isolation logique par agent

9.3 Indexation

Index automatiques :

- `id` (unique)

- `agent_id` (partition key)

Index personnalisés :

```
{
  "indexingPolicy": {
    "automatic": true,
    "indexingMode": "consistent",
    "includedPaths": [
      {
        "path": "/agent_name/*"
      },
      {
        "path": "/status/*"
      },
      {
        "path": "/created_at/*"
      },
      {
        "path": "/country/*"
      }
    ],
    "excludedPaths": [
      {
        "path": "/system_prompt/*"
      }
    ]
  }
}
```

9.4 Requêtes Optimisées

9.4.1 Liste des Agents par Status

```
def list_avatars_by_status(status):
    """Liste les avatars par status"""
    container = get_avatar_container()

    query = """
        SELECT * FROM c
        WHERE c.status = @status
        ORDER BY c.created_at DESC
    """

    parameters = [
        {"name": "@status", "value": status}
    ]

    items = list(container.query_items(
        query=query,
```

```
        parameters=parameters,  
        enable_cross_partition_query=True  
    ))  
  
    return items
```

9.4.2 Recherche par Nom

```
def search_avatars_by_name(search_term):  
    """Recherche d'avatars par nom"""  
    container = get_avatar_container()  
  
    query = """  
        SELECT * FROM c  
        WHERE CONTAINS(LOWER(c.agent_name), LOWER(@search))  
        ORDER BY c.agent_name  
    """  
  
    parameters = [  
        {"name": "@search", "value": search_term}  
    ]  
  
    items = list(container.query_items(  
        query=query,  
        parameters=parameters,  
        enable_cross_partition_query=True  
    ))  
  
    return items
```

9.4.3 Statistiques Globales

```
def get_avatar_statistics():  
    """Récupère les statistiques des avatars"""  
    container = get_avatar_container()  
  
    # Compte par status  
    query_status = """  
        SELECT c.status, COUNT(1) as count  
        FROM c  
        GROUP BY c.status  
    """  
  
    status_counts = list(container.query_items(  
        query=query_status,  
        enable_cross_partition_query=True  
    ))
```

```

# Compte par pays
query_country = """
    SELECT c.country, COUNT(1) as count
    FROM c
    GROUP BY c.country
"""

country_counts = list(container.query_items(
    query=query_country,
    enable_cross_partition_query=True
))

return {
    "by_status": status_counts,
    "by_country": country_counts
}

```

9.5 Gestion des Erreurs Cosmos DB

```

from azure.cosmos import exceptions

def safe_read_avatar(agent_id):
    """Lecture sécurisée avec gestion d'erreurs"""
    try:
        container = get_avatar_container()

        item = container.read_item(
            item=agent_id,
            partition_key=agent_id
        )

        return item

    except exceptions.CosmosResourceNotFoundError:
        logger.warning(f"Avatar {agent_id} introuvable")
        return None

    except exceptions.CosmosHttpResponseError as e:
        if e.status_code == 429: # Too Many Requests
            logger.error("Limite de débit Cosmos DB atteinte")
            # Retry avec backoff exponentiel
            time.sleep(1)
            return safe_read_avatar(agent_id)
        else:
            logger.exception("Erreur HTTP Cosmos DB")
            raise

    except Exception as e:
        logger.exception("Erreur inattendue Cosmos DB")
        raise

```


10. Sécurité et Authentification

10.1 Gestion des Secrets

Fichier : `.env` (NON COMMITÉ en production)

```
# Azure OpenAI
AZURE_OPENAI_API_KEY=sk-...
AZURE_OPENAI_ENDPOINT=https://...
AZURE_OPENAI_DEPLOYMENT_NAME=gpt-4-turbo

# Cosmos DB
COSMOS_DB_ENDPOINT=https://...
COSMOS_DB_KEY=...

# Azure Storage
AZURE_STORAGE_CONNECTION_STRING=...

# Azure Communication
AZURE_COMMUNICATION_CONNECTION_STRING=...
AZURE_EMAIL_SENDER=noreply@waka.com

# Azure Speech
AZURE_SPEECH_ENDPOINT=https://...
AZURE_SPEECH_KEY=...

# WeatherAPI
WEATHER_API_KEY=...

# Flask
SECRET_KEY=...
FLASK_ENV=production
```

10.2 Sécurisation des Endpoints API

```
from functools import wraps
from flask import request, jsonify

def require_api_key(f):
    """Décorateur pour valider l'API key"""
    @wraps(f)
    def decorated_function(*args, **kwargs):
        api_key = request.headers.get('X-API-Key')

        if not api_key:
            return jsonify({
                'error': 'API key manquante'
            }), 401
    return decorated_function
```

```

        expected_key = os.getenv('API_KEY')

        if api_key != expected_key:
            return jsonify({
                'error': 'API key invalide'
            }), 403

        return f(*args, **kwargs)

    return decorated_function

# Utilisation
@avatar_bp.route('/api/sensitive-endpoint', methods=['POST'])
@require_api_key
def sensitive_endpoint():
    # Code sécurisé
    pass

```

10.3 Validation des Entrées

```

from werkzeug.utils import secure_filename
import re

def validate_agent_name(name):
    """Valide le nom d'agent"""
    if not name or len(name) < 3:
        raise ValueError("Nom trop court (min 3 caractères)")

    if len(name) > 100:
        raise ValueError("Nom trop long (max 100 caractères)")

    # Caractères autorisés: lettres, chiffres, espaces, tirets
    if not re.match(r'^[a-zA-Z0-9\s\-\éeèââôûç]+$ ', name):
        raise ValueError("Caractères invalides dans le nom")

    return name.strip()

def validate_email(email):
    """Valide une adresse email"""
    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'

    if not re.match(pattern, email):
        raise ValueError("Format email invalide")

    return email.lower()

def sanitize_prompt(prompt):
    """Nettoie et valide le prompt système"""
    if not prompt or len(prompt) < 50:
        raise ValueError("Prompt trop court (min 50 caractères)")

```

```

if len(prompt) > 10000:
    raise ValueError("Prompt trop long (max 10000 caractères)")

# Supprimer les scripts potentiellement malveillants
prompt = re.sub(r'<script[^>]*>.*?</script>', '', prompt,
               flags=re.IGNORECASE | re.DOTALL)

return prompt.strip()

```

10.4 Rate Limiting

```

from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

limiter = Limiter(
    app,
    key_func=get_remote_address,
    default_limits=["200 per day", "50 per hour"]
)

@avatar_bp.route('/api/<agent_id>/generate_prompt', methods=['POST'])
@limiter.limit("10 per hour")
def generate_prompt_api(agent_id):
    # Limité à 10 générations par heure
    pass

@avatar_bp.route('/api/avatars', methods=['GET'])
@limiter.limit("100 per hour")
def list_avatars():
    # Limité à 100 requêtes par heure
    pass

```

10.5 Protection CORS

```

from flask_cors import CORS

# Configuration restrictive en production
CORS(app, resources={
    r"/api/*": {
        "origins": ["https://waka.azurewebsites.net"],
        "methods": ["GET", "POST", "PUT", "DELETE"],
        "allow_headers": ["Content-Type", "Authorization"],
        "max_age": 3600
    }
})

```

11. Optimisations et Performance

11.1 Mise en Cache

```
from flask_caching import Cache

cache = Cache(app, config={
    'CACHE_TYPE': 'simple',
    'CACHE_DEFAULT_TIMEOUT': 300
})

@avatar_bp.route('/api/avatars', methods=['GET'])
@cache.cached(timeout=60)
def list_avatars_cached():
    """Liste des avatars avec cache de 60 secondes"""
    avatars = list_avatar_configs()
    return jsonify(avatars)
```

11.2 Pagination

```
def list_avatars_paginated(page=1, page_size=20):
    """Liste paginée des avatars"""
    container = get_avatar_container()

    offset = (page - 1) * page_size

    query = f"""
        SELECT * FROM c
        ORDER BY c.created_at DESC
        OFFSET {offset} LIMIT {page_size}
    """

    items = list(container.query_items(
        query=query,
        enable_cross_partition_query=True
    ))

    # Compte total (cached)
    total_query = "SELECT VALUE COUNT(1) FROM c"
    total = list(container.query_items(
        query=total_query,
        enable_cross_partition_query=True
    ))[0]

    return {
        "items": items,
        "page": page,
        "page_size": page_size,
        "total": total,
        "pages": (total + page_size - 1) // page_size
    }
```

11.3 Compression Gzip

```
from flask_compress import Compress

Compress(app)
```

11.4 Lazy Loading des Avatars

```
// Chargement à la demande des images d'avatar
function lazyLoadAvatars() {
  const observer = new IntersectionObserver((entries) => {
    entries.forEach(entry => {
      if (entry.isIntersecting) {
        const img = entry.target;
        const src = img.dataset.src;

        if (src) {
          img.src = src;
          img.removeAttribute('data-src');
          observer.unobserve(img);
        }
      }
    });
  });

  document.querySelectorAll('img[data-src]').forEach(img => {
    observer.observe(img);
  });
}
```

11.5 Minification des Assets

Configuration de build :

```
# config.py

if os.getenv('FLASK_ENV') == 'production':
  # Minifier JS
  MINIFY_JS = True

  # Minifier CSS
  MINIFY_CSS = True

  # Combiner les assets
  BUNDLE_ASSETS = True
```

12. Défis Techniques Résolus

12.1 Bug Azure Avatar - Séparation Audio/Vidéo

Problème : L'API Azure Avatar Live avait un bug critique où l'utilisation d'un seul transceiver pour audio+vidéo causait des déconnexions aléatoires.

Solution : Créer des transceivers séparés

```
// ✗ AVANT (causait des bugs)
peerConnection.addTransceiver('video', {direction: 'recvonly'});

// ✔ APRÈS (fonctionne)
peerConnection.addTransceiver('audio', {direction: 'recvonly'});
peerConnection.addTransceiver('video', {direction: 'recvonly'});
```

Temps de résolution : 8 heures de debugging

12.2 Avatar Selection Bug - Valeur "lisa" en Dur

Problème : Quel que soit l'avatar sélectionné, la valeur "lisa" était toujours sauvegardée dans Cosmos DB.

Cause :

1. Champ caché avec `value="lisa"` en dur
2. Pas de handler de clic sur les cartes d'avatar

Solution :

```
// 1. Champ caché sans valeur par défaut
<input type="hidden" id="avatarCharacter"
      name="avatar_character" value="">

// 2. Handler de clic ajouté
card.addEventListener('click', async function() {
  // Mise à jour visuelle
  grid.querySelectorAll('.avatar-card').forEach(c =>
    c.classList.remove('selected')
  );
  this.classList.add('selected');

  // Mise à jour du champ caché
  avatarCharacterInput.value = character;

  // Sauvegarde immédiate dans Cosmos DB
  await fetch(`/avatar/api/${agentId}/update_avatar_character`, {
    method: 'POST',
    body: JSON.stringify({character, style})
  });
});
```

Temps de résolution : 4 heures

12.3 WebSocket Disconnection Issues

Problème : Déconnexions fréquentes du WebSocket après 30-60 secondes.

Cause : Pas de keep-alive / ping-pong

Solution :

```
// Ping automatique toutes les 30 secondes
setInterval(() => {
  if (socket.readyState === WebSocket.OPEN) {
    socket.send(JSON.stringify({type: 'ping'}));
  }
}, 30000);

// Reconnexion automatique
socket.onclose = () => {
  console.log('🚫 WebSocket déconnecté, reconnexion...');

  setTimeout(() => {
    connectWebSocket();
  }, 1000);
};
```

Temps de résolution : 3 heures

12.4 ICE Servers Configuration

Problème : WebRTC ne se connectait pas (pas de flux vidéo).

Cause : ICE servers non configurés correctement

Solution : Récupérer les ICE servers depuis Azure lors de la création de session

```
// Récupération des ICE servers depuis session.updated
socket.onmessage = (event) => {
  const message = JSON.parse(event.data);

  if (message.type === 'session.updated') {
    const iceServers = message.session.ice_servers;

    // Créer RTCPeerConnection avec ces servers
    peerConnection = new RTCPeerConnection({
      iceServers: iceServers
    });
  }
};
```

Temps de résolution : 6 heures

12.5 Avatars Sans Styles

Problème : Certains avatars n'ont pas de styles définis, causant des erreurs JavaScript.

Solution : Fallback gracieux

```
function updateAvatarStyles(character) {
  const styles = avatarStyles[character];

  if (!styles || styles.length === 0) {
    // Cache la section style
    styleGroup.style.display = 'none';

    // Valeur par défaut
    styleInput.value = 'casual-sitting';

    console.log(`⚠ Style par défaut pour ${character}`);
    return;
  }

  // Affichage normal des styles
  styleGroup.style.display = 'block';
  renderStyles(styles);
}
```

Temps de résolution : 2 heures

12.6 Prompt Tools Injection

Problème : Le modèle n'appelait pas les outils car ils n'étaient pas documentés dans le prompt.

Solution : Injection automatique de la documentation complète des outils

```
# Après génération du prompt par GPT-4
generated_prompt = generate_with_gpt4(instructions)

# Ajouter la section outils
tools_section = build_tools_documentation(selected_tools)

final_prompt = generated_prompt + tools_section
```

Temps de résolution : 5 heures

13. Tests et Validation

13.1 Tests Unitaires

```
# tests/test_avatar_routes.py

import unittest
from app import app

class TestAvatarRoutes(unittest.TestCase):

    def setUp(self):
        self.app = app.test_client()
        self.app.testing = True

    def test_gallery_page(self):
        """Test de la page galerie"""
        response = self.app.get('/avatar/gallery')
        self.assertEqual(response.status_code, 200)
        self.assertIn(b'Mes Agents Avatar', response.data)

    def test_create_avatar_step1(self):
        """Test de création d'avatar étape 1"""
        response = self.app.post('/avatar/step1', data={
            'agent_name': 'Test Agent',
            'description': 'Agent de test',
            'country': 'Burkina Faso'
        })

        self.assertEqual(response.status_code, 302) # Redirect

    def test_update_avatar_character_api(self):
        """Test de l'API de mise à jour character"""
        response = self.app.post(
            '/avatar/api/test-id/update_avatar_character',
            json={
                'character': 'harry',
                'style': 'formal-standing'
            }
        )

        self.assertEqual(response.status_code, 200)
        data = response.get_json()
        self.assertTrue(data['success'])
        self.assertEqual(data['character'], 'harry')

if __name__ == '__main__':
    unittest.main()
```

13.2 Tests d'Intégration

```
# tests/test_integration.py

def test_complete_avatar_workflow():
    """Test du workflow complet de création d'avatar"""

    # Étape 1 : Créer l'agent
    response1 = client.post('/avatar/step1', data={
        'agent_name': 'Integration Test',
        'description': 'Test complet',
        'country': 'Burkina Faso'
    })

    agent_id = extract_agent_id(response1.location)
    assert agent_id is not None

    # Étape 2 : Configurer avatar et voix
    response2 = client.post(f'/avatar/step2/{agent_id}', data={
        'avatar_character': 'lisa',
        'avatar_style': 'casual-sitting',
        'voice_type': 'azure',
        'voice_id': 'fr-FR-DeniseNeural'
    })

    assert response2.status_code == 302

    # Étape 3 : Sélectionner outils
    response3 = client.post(f'/avatar/step3/{agent_id}', data={
        'tools[]': ['get_weather_forecast', 'send_email']
    })

    assert response3.status_code == 302

    # Étape 4 : Définir le prompt
    response4 = client.post(f'/avatar/step4/{agent_id}', data={
        'system_prompt': 'Tu es un assistant...'
    })

    assert response4.status_code == 302

    # Vérifier dans la base de données
    config = get_avatar_config(agent_id)
    assert config is not None
    assert config['status'] == 'completed'
    assert config['current_step'] == 4
```

13.3 Tests de Charge

```
# tests/test_load.py

from locust import HttpUser, task, between
```

```

class AvatarUser(HttpUser):
    wait_time = between(1, 3)

    @task(3)
    def view_gallery(self):
        """Visite de la galerie (tâche fréquente)"""
        self.client.get('/avatar/gallery')

    @task(2)
    def view_avatar_details(self):
        """Visualisation des détails d'un avatar"""
        self.client.get('/avatar/call/test-agent-id')

    @task(1)
    def create_avatar(self):
        """Création d'avatar (tâche moins fréquente)"""
        self.client.post('/avatar/step1', data={
            'agent_name': 'Load Test Agent',
            'description': 'Test de charge',
            'country': 'Burkina Faso'
        })

# Exécution:
# locust -f test_load.py --host=https://waka.azurewebsites.net

```

Résultats attendus :

- 100 utilisateurs simultanés
- Temps de réponse < 2s (95e percentile)
- 0% d'erreurs

14. Déploiement et CI/CD

14.1 Configuration Azure Web App

Fichiers de configuration :

— Procfile	# Configuration Gunicorn
— requirements.txt	# Dépendances Python
— startup.txt	# Script de démarrage
— .gitignore	# Fichiers ignorés
— .env	# Variables d'environnement

Procfile :

```

web: gunicorn app:app --bind 0.0.0.0:$PORT --timeout 120 --workers 4 --threads 2 -
-worker-class sync

```

startup.txt :

```
gunicorn app:app --bind 0.0.0.0:8000 --timeout 120 --workers 4 --threads 2 --  
worker-class sync --access-logfile - --error-logfile -
```

14.2 Déploiement via Git

```
# Ajouter le remote Azure  
git remote add azure https://$waka:  
<password>@waka.scm.azurewebsites.net:443/waka.git  
  
# Commit et push  
git add .  
git commit -m "Déploiement Waka Voice Avatar"  
git push azure master
```

Processus de build Azure :

1. Détection de Python 3.14
2. Installation des dépendances (`pip install -r requirements.txt`)
3. Configuration de Gunicorn
4. Démarrage de l'application

Temps de déploiement : ~30 secondes

14.3 Variables d'Environnement Azure

Configuration dans le portail Azure :

- **App Service** → **Configuration** → **Application settings**

Ajouter toutes les variables du `.env` :

- AZURE_OPENAI_API_KEY
- AZURE_OPENAI_ENDPOINT
- COSMOS_DB_ENDPOINT
- COSMOS_DB_KEY
- etc.

14.4 Logs et Monitoring

Activation des logs :

```
az webapp log tail --name waka --resource-group waka-rg
```

Application Insights :

```
from applicationinsights.flask.ext import AppInsights

appinsights = AppInsights(app)

# Tracking automatique des requêtes, exceptions, dépendances
```

15. Maintenance et Évolution

15.1 Roadmap Technique

Version 2.0 (Q1 2026) :

- ☐ Support multi-langues (anglais, mooré, dioula)
- ☐ Avatars personnalisés (upload de modèles 3D)
- ☐ Intégration téléphonique (appels entrants/sortants)
- ☐ Analytics avancés (dashboards, métriques)

Version 2.1 (Q2 2026) :

- ☐ Mode hors ligne (PWA)
- ☐ Authentification utilisateurs
- ☐ API publique pour développeurs
- ☐ Marketplace d'outils

15.2 Améliorations de Performance

Optimisations identifiées :

- Mise en cache Redis pour Cosmos DB queries
- CDN pour assets statiques
- Compression d'images avatars
- Lazy loading complet

Gains attendus :

- Réduction temps de chargement : -40%
- Réduction coûts Cosmos DB : -30%
- Amélioration UX : Score Lighthouse > 90

15.3 Debt Technique

Items identifiés :

1. Refactoring `avatar_routes.py` (trop volumineux - 1200 lignes)
2. Tests unitaires à compléter (couverture actuelle : 60%)
3. Documentation API (Swagger/OpenAPI)
4. Migration vers Flask 3.1 (dernière version)

Estimation : 20 heures

15.4 Monitoring Production

Métriques clés à surveiller :

- Temps de réponse API (< 2s p95)
- Taux d'erreur (< 1%)
- Disponibilité (> 99.5%)
- Utilisation Cosmos DB (RU/s)
- Durée sessions WebSocket
- Coûts Azure (budget mensuel)

Alertes configurées :

- Erreur rate > 5% → Email + SMS
- Temps réponse > 5s → Email
- Crash application → Email + SMS
- Budget dépassé → Email

Conclusion

Synthèse du Projet

Le système Avatar AI de Waka Voice Burkina représente **140 heures de développement intensif**, produisant une plateforme complète et robuste pour la création d'agents conversationnels avec avatars vidéo.

Chiffres clés :

- **52,676 lignes de code** (125 fichiers)
- **24 outils métiers** intégrés
- **4 étapes de configuration** utilisateur
- **3 services Azure majeurs** (OpenAI, Cosmos DB, Avatar Live)
- **WebRTC + WebSocket** pour temps réel
- **Architecture modulaire** (Flask Blueprints)

Points Forts Techniques

1. **Architecture Modulaire** : Séparation claire des responsabilités
2. **Scalabilité** : Cosmos DB distribué, Unicorn multi-workers
3. **Robustesse** : Gestion complète des erreurs, fallbacks gracieux
4. **Temps Réel** : WebSocket + WebRTC pour expérience fluide
5. **Extensibilité** : Système d'outils facilement extensible

Défis Surmontés

- Bugs complexes Azure Avatar (séparation audio/vidéo)
- Gestion états WebSocket/WebRTC
- Optimisation requêtes Cosmos DB

- Intégration 24 outils différents
- Configuration multi-étapes robuste

Impact Business

Cette solution permet à Waka Voice Burkina de :

- Créer des agents conversationnels en 10 minutes
- Offrir une expérience utilisateur premium (avatar vidéo)
- S'adapter à différents métiers (24 outils)
- Évoluer facilement (architecture modulaire)

ROI estimé : Réduction de 80% du temps de configuration d'agents

Auteur : Équipe Technique Waka **Contact** : tech@waka.com **Version** : 1.0 **Date** : 19 Novembre 2025

Cette documentation représente fidèlement 140 heures de travail technique intensif sur le système Avatar AI.